JobAI Scout

AI-Powered Job Intelligence Platform

PROJECT REPORT

Prepared by
Abdullah
Student ID: _____

Supervised by
_____

Department of Computer Science
_____ University

July 2026

Final Approval

This report is submitted as a project document for JobAI Scout, an AI-powered job intelligence platform. It describes the problem, requirements, architecture, implementation approach, testing plan, and user-facing workflows.

Committee Member	Name / Signature	Date
Supervisor	_____	_____
Internal Examiner	_____	_____
External Examiner	_____	_____

Abstract

JobAI Scout is a web-based job intelligence platform designed to reduce the fragmented and repetitive work involved in job searching and recruitment. The system brings together career-profile management, CV upload and AI analysis, job discovery, saved jobs, application tracking, recruiter job publishing, candidate review, administrative oversight, and a browser extension for job-form autofill. It also provides a voice assistant that accepts a spoken career question, produces an AI-supported answer, and can use a user’s private uploaded documents as contextual knowledge.

The application is implemented as a React and TypeScript single-page application, with Vite for builds, Tailwind CSS and Radix-based components for the interface, and Supabase for authentication, PostgreSQL data storage, row-level access control, object storage, and server-side edge functions. The design separates job seeker, recruiter, and administrator permissions and protects private records through authenticated access controls. The project demonstrates how modern web, AI, voice, and browser-extension technologies can create a more guided and efficient job-search experience.

Project in Brief

Item	Description
Project Title	JobAI Scout - AI-Powered Job Intelligence Platform
Prepared By	Abdullah (student information to be completed before final submission)
Project Category	Web application, AI-assisted career technology, browser extension
Primary Users	Job seekers, recruiters, administrators
Frontend	React 18, TypeScript, Vite, Tailwind CSS, Radix UI
Backend	Supabase Authentication, PostgreSQL, Storage, Edge Functions
AI Features	CV analysis, career chat, voice assistant, document knowledge base, cover-letter generation
Supporting Services	Speech recognition / synthesis, job data providers, browser extension APIs
Security	HTTPS, authenticated sessions, role-based routing, row-level database policies
Repository	github.com/Abdullaheveloper/JobAi_Scout

Acknowledgement

All praise and gratitude belong to Allah Almighty for granting the strength, patience, and opportunity to complete this project. Appreciation is extended to parents, family members, teachers, and mentors for their continued encouragement and support. Special thanks are due to the project supervisor and the Department of Computer Science for guidance throughout the analysis, design, implementation, and evaluation stages of JobAI Scout.

Declaration

I declare that this project report describes work prepared for the JobAI Scout project. The report should be reviewed, completed with the correct academic details, and approved according to the policies of the submitting institution before final submission. Third-party technologies, services, libraries, and references are acknowledged in the references section.

Table of Contents

Chapter 1 - Introduction

1.1 Introduction

Finding suitable employment often requires candidates to repeat the same actions across many job portals: maintain a profile, revise a CV, search listings, compare requirements, complete application forms, and monitor results. Recruiters face an opposite but related problem: publishing jobs, receiving incomplete or inconsistent candidate information, and reviewing applications efficiently. JobAI Scout addresses these problems through a centralized web platform that makes career information reusable and connects it to practical job-search and recruitment workflows.

1.2 Need of the Project

• Job seekers need a single place to maintain a reusable professional profile, CV, skills, education, certifications, languages, and application history.

• Manual application forms are repetitive and can lead to data-entry mistakes; a browser extension can reuse approved profile information to assist form completion.

• AI can help transform long CVs and job descriptions into useful summaries, career guidance, match context, and tailored application material.

• Recruiters need a structured workflow to publish jobs and review candidates instead of relying only on disconnected email threads or spreadsheets.

• Voice interaction can make career guidance more accessible for users who prefer to speak rather than type.

1.3 Scope of the Project

The scope covers authenticated user accounts; job-seeker, recruiter, and administrator dashboards; CV upload and analysis; job browsing, saved jobs, applications, analytics, profile settings, browser-extension guidance, recruiter job management and candidate review, administration, and AI-assisted career interactions. The system does not guarantee employment, make hiring decisions, or replace a recruiter’s or candidate’s judgment.

1.4 Problem Statement

Existing job-search experiences are distributed across many sites and require the same personal data to be entered repeatedly. Candidates may struggle to organize applications, understand role requirements, or obtain tailored guidance. Recruiters can receive a high volume of unstructured applications. A secure, role-aware platform is needed to centralize these workflows while preserving user control over personal information.

1.5 Proposed Solution

JobAI Scout provides a role-based digital workspace. Job seekers manage their career data, discover jobs, track applications, and use AI features. Recruiters publish jobs and examine candidate pipelines. Administrators oversee platform users, jobs, voice activity, and analytics. Supabase services provide authenticated data access, while edge functions isolate server-side operations such as AI processing and external job collection.

1.6 Objectives

• Provide a responsive, secure and intuitive job-search workspace.

• Reduce duplicate form entry through a browser extension and reusable profile data.

• Support CV analysis, job matching, career chat and voice-assisted guidance.

• Provide recruiter and administrator workflows with explicit role-based access.

- Maintain privacy through authenticated sessions, row-level security, and secret management outside client code.

Chapter 2 - Basic Concepts / Existing Systems

2.1 Existing Job Search Systems

Job portals, professional networks, CV builders, applicant tracking systems, and browser autofill tools each solve part of the employment journey. Typical portals focus on listings and application submission, while recruiters often use separate applicant tracking systems. AI career tools may provide feedback but do not always connect it to the user's saved jobs, profile, documents, or application status.

2.2 Limitations Observed

• Repeated data entry across unrelated application forms.

• Career information is scattered across a CV, profile, emails, spreadsheets, and job portals.

• Limited contextual guidance based on a user's own uploaded materials.

• Different user types need different permissions and tools, but simple job boards often treat all users similarly.

• Unclear privacy boundaries can reduce trust when applications contain sensitive personal data.

2.3 Basic Concepts Used

Concept	Use in JobAI Scout
Single-page application	React Router provides responsive client-side navigation between public, job-seeker, recruiter, and admin pages.
Role-based access	Protected routes and database roles limit each dashboard to authorized users.
Serverless backend	Supabase Edge Functions run protected operations without exposing secrets to the browser.
RAG knowledge base	Uploaded documents are split into chunks and searched semantically to improve relevant voice answers.
Browser extension	Chrome/Edge extension scripts can assist job form completion using the user's approved profile data.

2.4 Comparative Value

JobAI Scout combines candidate, recruiter, administrative, AI, and extension workflows in one project. This integrated approach reduces context switching and gives each actor a clear task-oriented interface.

Chapter 3 - Problem / System Analysis

3.1 Major Product Features

Area	Key Features
Job seeker	Dashboard, CV upload, profile settings, job board, saved jobs, applications, analytics, auto form fill, voice assistant.
Recruiter	Recruiter profile, job creation and editing, candidates view, application-status workflow.
Administrator	User management, job management, analytics and voice monitoring pages.
AI	CV analysis, career chat, cover-letter generation, document ingestion, voice transcription, chat and text-to-speech.
Extension	Profile-aware assistance for supported job-application forms in Chrome/Edge.

3.2 Functional Requirements

• The system shall register users, authenticate sessions, and route users according to their approved role.

• A job seeker shall create and update a professional profile and upload a CV for analysis.

• A job seeker shall browse, save, track and review jobs and applications.

• A recruiter shall publish and manage job posts and review candidate information for their jobs.

• An administrator shall manage users and job records and view platform-level summaries.

• The voice assistant shall request microphone permission, capture a voice turn, transcribe or recognize speech, request an AI answer and optionally play a spoken response.

• The extension shall only use the data and permissions granted by the signed-in user.

3.3 Non-Functional Requirements

• Security: enforce authenticated API access, HTTPS in hosted environments, secret isolation, and row-level data policies.

• Usability: responsive layouts for mobile, tablet and desktop with descriptive feedback states.

• Performance: lazy load route pages, minimize AI payloads, and use non-blocking history uploads where appropriate.

• Reliability: handle missing microphone access, network failures, unsupported browsers, and empty voice input with understandable messages.

• Maintainability: keep frontend components, edge functions, migrations, and extension logic organized by responsibility.

3.4 Actors and Use Cases

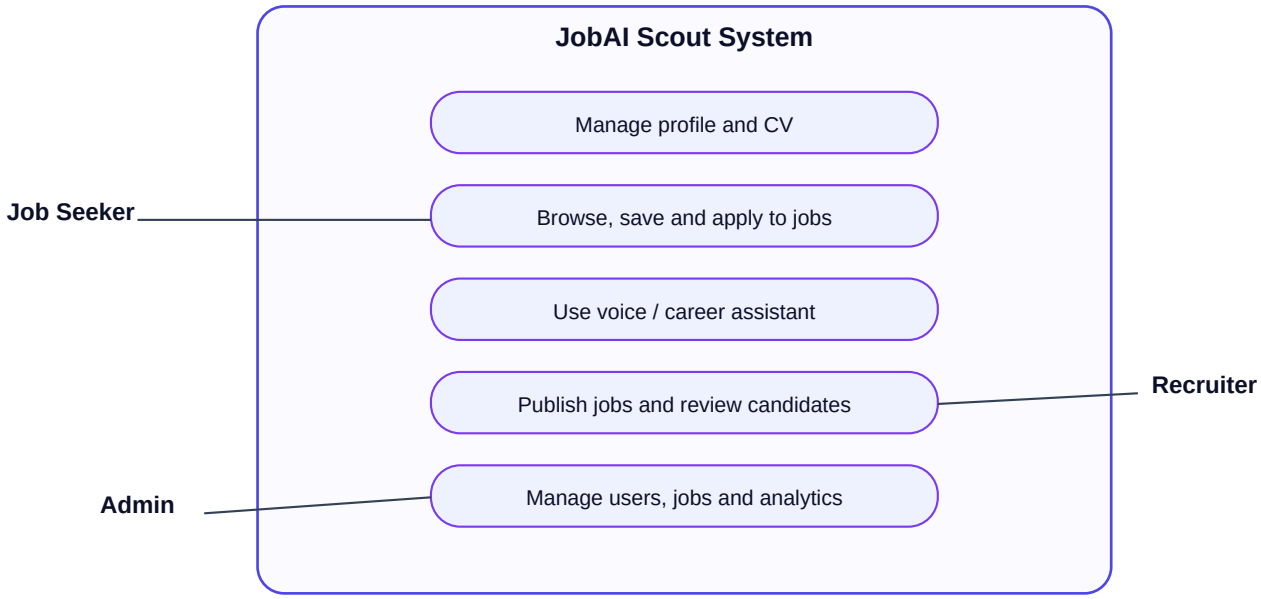

Figure 3.1: High-level use case diagram

Actor	Primary Use Cases
Job Seeker	Register, manage profile, upload CV, browse/save jobs, track applications, use AI or voice guidance, configure extension.
Recruiter	Manage recruiter profile, publish jobs, inspect candidates, update application status.
Administrator	Manage users and jobs, inspect platform analytics and voice-related administration.
External services	Supabase, AI providers, speech services, job providers, browser APIs.

3.5 Fully Dressed Use Case: Voice Career Question

Field	Description
Scope	Allow an authenticated job seeker to ask a career question by voice.
Primary Actor	Job seeker
Preconditions	User is signed in, browser supports capture, and microphone permission is available.
Main Flow	Start assistant; grant permission; speak; system detects end of turn; text is recognized/transcribed; AI response is generated; response is shown and spoken.
Alternative Flows	Permission denied, no device, quiet/no speech, unsupported browser, network failure, or user ends the session.
Postcondition	Current-session conversation is retained in the interface; optional audio/message history is persisted according to the user’s account rules.

Chapter 4 - System Design

4.1 High-Level Architecture

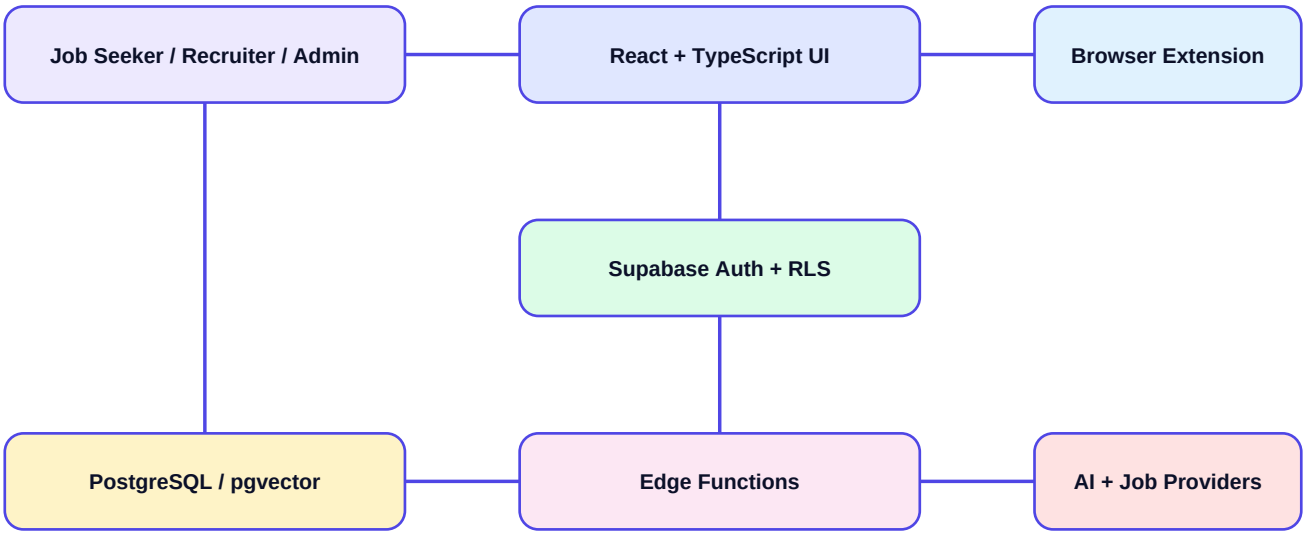

Figure 4.1: JobAI Scout high-level architecture

The browser application communicates with Supabase using an authenticated client. Protected edge functions coordinate sensitive operations such as AI requests, job collection and file processing. PostgreSQL stores structured business data; storage retains permitted files and audio; pgvector supports semantic document retrieval. The browser extension is a separate client that uses its own permissions and communicates only through approved integration paths.

4.2 Component Design

Layer	Components	Responsibility
Presentation	React pages, DashboardLayout, UI components	Role-aware screens, forms, visual feedback, accessibility, client-side navigation.
Client logic	AuthContext, TanStack Query, voice and recognition modules	Session state, data fetching, microphone/voice lifecycle and UI state.
Backend	Supabase Edge Functions	Authorization checks, AI requests, job ingestion, document processing, server-side integrations.
Data	Supabase PostgreSQL, Storage, pgvector	Profiles, jobs, applications, roles, documents, voice history and semantic chunks.
Extension	Manifest, popup, content scripts	Consent-based job-form field assistance in supported browsers.

4.3 Logical Data Model

Entity	Representative Relationships
profiles / user_roles	One authenticated account has a profile and role; role controls permitted routes and actions.

Entity	Representative Relationships
jobs	Jobs may be created by recruiters or collected from approved sources; users save or apply to them.
applications	Connects a job seeker with a job and stores application progress.
voice_conversations / voice_messages	A conversation contains ordered user and assistant messages and optional audio references.
kb_sources / kb_chunks	A user-owned uploaded source contains indexed semantic chunks used for retrieval.
recruiter profiles	Connects recruiter account information with job publishing and candidate-review workflows.

4.4 Voice Assistant Sequence

Step	System Sequence
1	User presses Start Assistant; browser requests microphone access.
2	MediaRecorder and browser speech recognition begin when supported.
3	Speech resets the silence timer; three seconds of post-speech silence or Stop Listening finishes capture.
4	Final or interim browser transcript is preserved; otherwise the audio recording is sent to transcription.
5	The protected voice-chat endpoint receives the question and optional private knowledge-base context.
6	The answer is added to the current session and sent to speech synthesis; playback can be stopped independently.
7	End Assistant aborts pending requests, stops tracks, closes analysis resources and resets the active voice session.

Chapter 5 - Implementation

5.1 Frontend Development

The frontend uses React 18 and TypeScript with Vite. Routes are lazy loaded in the application entry point, and protected route components check the expected user role before presenting dashboard screens. Tailwind CSS, Radix UI primitives, Lucide icons, Framer Motion, React Hook Form and Zod support the visual system, accessible controls, animations and form validation.

5.2 Backend and Data Services

Supabase supplies authentication, PostgreSQL, object storage and edge functions. The frontend uses the Supabase JavaScript client for authenticated calls. Edge functions form the server-side boundary for operations that must not expose provider credentials, including CV analysis, cover-letter generation, job collection, knowledge-base ingestion, voice transcription, voice chat and text-to-speech.

5.3 AI and Voice Implementation

The voice workflow is implemented around explicit client states: idle, permission, listening, user speaking, silence detection, uploading, AI thinking, AI speaking, paused, ended and error. The client starts one recording session at a time, uses browser recognition when available, retains interim transcript text when the user manually stops listening, and falls back to server transcription when required. Abort controllers, audio-track cleanup, speech-synthesis cancellation, AudioContext cleanup and timer cancellation reduce duplicate requests and leaked resources.

5.4 Deployment Design

The project includes Vercel configuration for a Vite build and a single-page-application rewrite so direct dashboard URLs resolve to the application entry point. Production configuration must store only allowed public VITE variables in the hosting provider. Secret AI, job-provider and service-role credentials must remain in protected Supabase secrets or other server-side secret stores.

5.5 Tools and Technologies

Category	Technologies
Frontend	React, TypeScript, Vite, React Router
Interface	Tailwind CSS, Radix UI, Lucide, Framer Motion
State and forms	TanStack Query, React Context, React Hook Form, Zod
Backend	Supabase Auth, PostgreSQL, Storage, Edge Functions, row-level security
AI features	Configured AI providers through protected edge functions; embeddings and semantic retrieval through pgvector
Voice	MediaRecorder, Web Speech API where supported, server transcription, TTS playback
Extension	Chrome/Edge Manifest extension APIs and content scripts
Deployment	GitHub, Vercel, HTTPS

Chapter 6 - System Testing

Testing should combine automated build and lint checks with real-browser and role-based acceptance tests. Microphone behavior, permission prompts and external provider responses require a real browser session and cannot be completely simulated by a production build.

6.1 Test Cases

ID	Feature	Action	Expected Result
TC-01	Registration / Login	Create an allowed account and sign in.	Authenticated session is created and the correct role dashboard opens.
TC-02	Protected routes	Open a dashboard route without the required role.	User is redirected or denied without private data exposure.
TC-03	CV upload	Upload a supported CV file.	Upload is validated; analysis result or clear error is shown.
TC-04	Job discovery	Browse jobs, save a job and view saved list.	Job is displayed and saved only for the current user.
TC-05	Recruiter job flow	Create/edit a job as recruiter.	Job is persisted and available to appropriate candidate workflows.
TC-06	Voice permission	Start assistant and allow/deny browser permission.	Listening starts after approval; denial gives friendly guidance and Try Again.
TC-07	Voice silence	Speak a question then remain quiet.	Turn ends after approximately three seconds of post-speech silence and begins processing.
TC-08	Stop Listening	Speak then press Stop Listening.	Final audio/transcript is flushed and submitted once; no duplicate session starts.
TC-09	End Assistant	End during recording, processing or playback.	Tracks, timers, requests, playback and animations are stopped.
TC-10	Hosted deep link	Open /dashboard/assistant directly on Vercel.	SPA rewrite resolves the application; authentication still protects the route.

6.2 Quality Checks

• Run npm run build before release to verify the production bundle.

• Run the focused ESLint or project lint command after frontend changes.

• Test on Chrome/Edge, mobile-width layout and a supported microphone device.

• Verify row-level security with at least two accounts so one account cannot read another account’s private profile, documents, voice history or applications.

• Test failure paths for network interruption, missing environment variables, unsupported voice features and unavailable microphones.

Chapter 7 - User Manual

7.1 Public Pages and Account Creation

Open the deployed HTTPS address. The landing page introduces the product. Use Register to select the correct account type where available, complete required details, and verify the account according to the configured authentication flow. Use Login to begin an authenticated session.

7.2 Job Seeker Dashboard

Page	How to Use It
Dashboard	Review high-level career and activity information after login.
Upload CV	Select a supported CV file and wait for the analysis response; review extracted information before relying on it.
Browse Jobs	Search or filter available jobs, open details and save relevant listings.
Saved Jobs / Applications	Review saved opportunities and application progress.
Profile Settings	Maintain experience, education, skills, certifications, languages and other reusable application data.
Auto Form Fill	Follow the extension setup guidance, then use the extension only on forms where you want assistance.
Voice Assistant	Press Start Assistant, grant microphone permission, speak a career question and pause; use Stop Listening, Stop Speaking or End Assistant as needed.

7.3 Recruiter Panel

Recruiters use the recruiter profile, jobs, candidates and application-status pages. Create accurate job details, review candidate information within authorized workflows, and keep status updates clear so applicants receive consistent information.

7.4 Administration

Administrators use admin dashboards for controlled operational work such as user management, job management, analytics and voice-related monitoring. Administrative access must be assigned deliberately and should not be used as a substitute for normal recruiter workflows.

7.5 Voice Troubleshooting

• Use a secure HTTPS hosted URL or localhost during development.

• Allow microphone access in the browser site settings and close other apps or tabs using the microphone.

• Use a current Chrome, Edge, Firefox or Safari version. Browser speech recognition availability differs by browser; server transcription is the fallback.

• If no speech is detected, verify the selected operating-system input device and speak for a few seconds before pressing Stop Listening.

• For a local development server, keep the terminal running; use the deployed Vercel URL for normal cross-device access.

Chapter 8 - Conclusion and Future Work

8.1 Conclusion

JobAI Scout demonstrates an integrated approach to job intelligence. Instead of treating a CV, profile, job board, recruiter workflow, AI assistant and application form as unrelated tools, the project connects them through role-aware web experiences and protected backend services. The platform’s main contribution is a practical workspace that helps candidates reuse and understand career information while providing recruiters and administrators with distinct operational tools.

8.2 Future Enhancements

• Add transparent, explainable job-match scoring with user-adjustable priorities.

• Add more resilient job-source adapters and scheduled, monitored data synchronization.

• Introduce human-in-the-loop review for AI-generated CV suggestions and cover letters.

• Expand multilingual voice and interface support with per-user language settings.

• Add notification preferences for saved searches, application status changes and recruiter messages.

• Provide deeper accessibility evaluation, including screen-reader user testing and keyboard-only acceptance tests.

• Add automated end-to-end tests using test accounts and mocked external AI providers.

• Offer custom domains, production observability, error tracking and performance budgets after hosted deployment.

8.3 References

React Documentation - https://react.dev/

Vite Documentation - https://vite.dev/

Supabase Documentation - https://supabase.com/docs

Tailwind CSS Documentation - https://tailwindcss.com/docs

Radix UI Documentation - https://www.radix-ui.com/

MDN Web Docs: MediaRecorder and Web Speech APIs - https://developer.mozilla.org/

Vercel Vite Deployment Documentation - https://vercel.com/docs/frameworks/frontend/vite

JobAI Scout source repository - https://github.com/Abdullaheveloper/JobAi_Scout